

```

2157     needs: dict[str, int] = {}
2158     for type_id in ("A", "B", "C"):
2159         typed = [x for x in trips if x["type_id"] == type_id]
2160         needs["{type_id}_uav"] = _max_overlap([(x["start_s"], x["return_s"]) for x in typed])
2161         needs["{type_id}_battery"] = _max_overlap(
2162             [(x["start_s"], x["battery_charge_complete_s"]) for x in typed]
2163         )
2164         needs["relay_uav"] = _max_overlap([(x["start_s"], x["available_s"]) for x in missions])
2165         needs["relay_pack"] = _max_overlap(
2166             [(x["start_s"], x["pack_charge_complete_s"]) for x in missions]
2167         )
2168         workload = sum(x["return_s"] - x["start_s"] for x in trips) + sum(
2169             x["available_s"] - x["start_s"] for x in missions
2170         )
2171         # The task statement defines group completion by the last transport or
2172         # copied relay mission returning to 001; turnaround/charging is a
2173         # resource constraint, not part of completion time.
2174         completion = max(
2175             [x["return_s"] for x in trips]
2176             + [x["return_s"] for x in missions]
2177             + [0.0]
2178         )
2179         return needs, workload, completion
2180
2181     central, _, _ = resources_for(set(services))
2182     output_rows: list[dict[str, Any]] = []
2183     summary_rows: list[dict[str, Any]] = []
2184     for k in (2, 3):
2185         best = None
2186         for labels_tail in itertools.product(range(k), repeat=len(blocks) - 1):
2187             labels = (0,) + labels_tail
2188             if set(labels) != set(range(k)):
2189                 continue
2190             groups = [set() for _ in range(k)]
2191             for block, label in zip(blocks, labels):
2192                 groups[label].update(block)
2193             group_values = [resources_for(group) for group in groups]
2194             total_need = {key: sum(value[0][key] for value in group_values) for key in inventory}
2195             shortage = sum(max(0, total_need[key] - inventory[key]) for key in inventory)
2196             total_configuration = sum(total_need.values())
2197             workloads = np.asarray([value[1] for value in group_values], dtype=float)
2198             cv = float(np.std(workloads, ddof=0) / np.mean(workloads))
2199             completion_range = max(value[2] for value in group_values) - min(value[2] for value in group_values)
2200         )
2201         canonical = tuple(tuple(sorted(group)) for group in groups)
2202         score = (shortage, total_configuration, cv, completion_range, canonical)
2203         if best is None or score < best[0]:
2204             best = (score, groups, group_values, total_need)
2205         if best is None:
2206             raise RuntimeError(f"No Q4 partition for K={k}")
2207         _, groups, group_values, total_need = best
2208         for group_index, (group, (needs, workload, completion)) in enumerate(zip(groups, group_values), start
2209 =1):
2210             output_rows.append(
2211                 {"K": k, "group": group_index, "services": ";".join(sorted(group)), **needs,
2212                  "workload_h": workload / 3600.0, "completion_s": completion}
2213             )
2214         for resource in inventory:
2215             summary_rows.append(
2216                 {"K": k, "resource": resource, "total_group_need": total_need[resource],
2217                  "inventory": inventory[resource],
2218                  "shortfall": max(0, total_need[resource] - inventory[resource]),
2219                  "central_need": central[resource],
2220                  "partition_redundancy": total_need[resource] - central[resource]}
2221             )
2222     return output_rows, summary_rows

```

C.5 代码与正文结果的对应关系

上述清单覆盖了正文主要数值结论的生成链条：问题一的 18 架次来自清单2；问题二的 22 架次及逐箱时限由清单3-5产生；问题三的 11 个中继架次和连续通信证据由清单6-9产生，其中清单7给出连续区间证书的核心；问题四的 1023 个两组候选与 57002